

RELATÓRIO AULA-007

Quiz

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

- 1- -4, 512
- 2- 3345
- 3- 3 52
- 4- 0
- 5- Padrão
- 6- True false true

PRECEDENCIA E ASSOCIATIVIDADE NA PRÁTICA

Precedência: a precedência determina qual operador será executado primeiro.

Associatividade: a associatividade determina a ordem de avaliação quando operadores de mesma precedência aparecem juntos.

Esquerda: * / % + -

Direita: ** (Python, JS, Ruby, Fortran), ++, --, menos unário

Visual Basic: ^ à esquerda!

Linguagens diferentes operam usando padrões diferentes em relação a ordem dos fatores, assim como características de sintaxe diferentes, essas diferenças podem mudar o resultado de uma operação matemática quando repetida em diferentes linguagens, um método de minimizar esse possível resultado inesperado é pelo uso de parênteses, que deixa a intenção explícita para o compilador e para o próximo leitor

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;          /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1();   /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

Operadores sobrecarregados ocorrem quando o mesmo símbolo possui vários significados.

EX: + soma inteiros, reais, e concatena

Conversões de tipos: alargamento, estreitamento e coerção

Alargamento/Estreitamento: quando o tipo de uma variável é convertido para um tipo maior, ou menor, através do uso de coerção (conversão automática de tipos feita pela linguagem para permitir uma operação)

```
int grande = 123456789;
float f = grande;          // alargamento implícito
System.out.println((int) f); // 123456792 !
```

Em JavaScript e PHP comparações devem ser feitas com ===, que não fazem coerção

Expressões relacionais e booleanas

Java, c#, Rust, Go, aceitam apenas IF booleano, enquanto C, Python, JavaScript aceitam qualquer valor

C - booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python - comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Avaliação em curto-circuito

Na avaliação em curto-circuito, uma expressão lógica é avaliada apenas até o ponto necessário para determinar seu resultado. Em `A && B`, se `A` for falso, `B` não é avaliado; em `A || B`, se `A` for verdadeiro, `B` não é avaliado.

Java · `&&` faz curto-circuito, `&` não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

Sentenças de atribuição

Sentença de atribuição é uma construção da linguagem que permite armazenar o resultado de uma expressão em uma variável ou posição de memória

Atribuição de modo Misto

Atribuição envolvendo tipos diferentes, podendo exigir conversão de tipos para que o valor possa ser armazenado corretamente.

Implícita: a própria linguagem realiza a conversão.

```
double d = 3;           // ok: alargamento
```

Explícita (casting): o programador solicita a conversão.

```
int i = (int) 3.7;      // 3 (cast explícito)
```

Estruturas de Controle

Sequência: os comandos são executados um após o outro, na ordem em que aparecem;

Seleção: permite escolher entre diferentes caminhos, dependendo de uma condição;

Iteração: permite repetir um conjunto de comandos enquanto uma condição for satisfeita ou durante determinado número de vezes.

Seleção de 2 caminhos

O `else` pendente acontece quando existem vários `if` aninhados e um `else` pode, aparentemente, estar associado a mais de um `if`

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else // de qual if é este else?
    result = 1;
printf("%d\n", result); // 1
```

O else pertence ao if mais próximo que ainda não possui um else

Seleção múltipla: do switch de C ao match

A seleção múltipla permite escolher um entre vários caminhos, dependendo do valor de uma expressão.

Em C, usamos switch quando queremos comparar uma expressão com vários valores possíveis.

No caso de o Break estar omitido, pode ocorrer fall-through e a execução continua no próximo case

Estruturas match, presentes em linguagens modernas, generalizam essa ideia ao permitir correspondência de padrões, tornando a seleção mais expressiva.

Laços controlados por contador

São estruturas de iteração nas quais a quantidade de repetições é determinada por um contador. O exemplo mais comum é o for.

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
    i += 2; // altera a contagem
} // 0 3 6 9

for (; i <= 100; ) // expressões opcionais
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):
    print(i, end=" ")
    i += 2 # NÃO muda nada
           # 0 1 2 3 4 5 6 7 8 9

for i in range(10, 0, -3): # 10 7 4 1
    ...
```

Em Python o for é usado principalmente para percorrer elementos de uma sequência ou iterável, não precisa de inicialização explícita, não possui condição como em C, a varável recebe automaticamente o próximo elemento

Laços lógicos, saídas e iteradores.

Laços lógicos

São laços em que a repetição é controlada por uma expressão lógica (condição), e não necessariamente por um contador. While, Do While

Saídas de laços

São comandos que permitem alterar o fluxo normal de uma repetição. Break, continue

Iteradores

Um iterador é um mecanismo usado para percorrer os elementos de uma coleção ou sequência, fornecendo um elemento por vez. For em Python, for each

Desvio incondicional: o goto

O goto é uma estrutura de desvio incondicional. Ele faz o programa saltar diretamente para um ponto específico do código, identificado por um rótulo (label).

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
    goto fail;           // duplicada: desvia SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;      // nunca é alcançada
fail:
    return err;          // 0 = "certificado válido"
}
```

goto não possui uma condição própria para realizar o salto.

Comandos protegidos de Dijkstra

Os comandos protegidos foram propostos por Edsger W. Dijkstra como uma forma de estruturar decisões e evitar o uso excessivo de desvios como goto. A ideia central é usar guardas (condições) para determinar quais comandos podem ser executados.

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Cada condição é uma guarda.

Nos comandos protegidos de Dijkstra, mais de uma guarda pode ser verdadeira. Quando isso acontece, uma das alternativas habilitadas pode ser escolhida, sem que a linguagem necessariamente determine qual será.

Enquanto alguma guarda for verdadeira, uma das alternativas habilitadas é executada. Quando nenhuma guarda for verdadeira, o laço termina.